TECHNICAL DOCUMENT

SkillVault
技术文档

跨 Windows / WSL 的 AI Agent 技能集中管理工具

项目：SkillVault v1.0.0

赛项：第九届传智杯 AI 创新应用挑战赛 · 个人赛

参赛编号：2035821

作者：陈乐铮

源码版本：github.com/nayutalot/SkillVault @ 2e3ede7

日期：2026 年 11 月

摘　要

本文档描述 SkillVault 的架构设计、意图控制策略与可复现性论证。SkillVault 是一个 Electron 桌面应用，用于在 Windows 11 与 WSL Ubuntu 24.04 双环境下，把散落在多个 AI Agent Harness 目录中的技能收拢到一个本地 Git 仓库，使其成为唯一真相源，各 Harness 目录以链接方式引用该仓库；全过程不依赖任何云端远程仓库。

项目业务代码由 AI 在人的评审与验收下生成，共 13,673 行生产代码与 567 个测试用例。文档重点阐述三个方面：一是以依赖注入、区域所有权标记与人类保留决策权构成的意图控制策略，如何在代码由 AI 生成的条件下把可信度问题转化为可客观检验的问题；二是双端各自扫描、共享单一 Git 仓库的架构设计，以及其中对 registry.json 单一真相源、Agent 自适应发现与目录链接状态判定的处理；三是可复现性论证，包括依赖锁定、环境变量最小化、三条独立验证路径，以及整理后的提交历史与 AI 对话快照之间的主题级关联。

文档同时明确列出当前版本尚未覆盖的范围，包括 SSH 与 Docker 远程目标仍为框架状态、部分操作链路未在真实环境验证、以及旧版 WSL 的字符编码兜底缺失等，以避免过度承诺。

目　录

第一章　项目背景与问题定义 1
1.1 真实痛点 1
1.2 核心设计原则 2
第二章　架构设计 2
2.1 总体架构 2
2.1.1 分层视角与组件视角 3
2.1.2 双宿主协同机制 4
2.2 分层结构与模块职责 5
2.2.1 接口层与安全边界 6
2.2.2 共享层的单一真相源设计 6
2.2.3 Agent 自适应发现机制 7
2.2.4 源码索引 8
2.3 关键机制：目录链接与链接状态判定 8
2.4 安全模型 9
2.4.1 配置写入与补偿恢复的边界 10
2.4.2 降级路径的信息保真 11
2.5 约束 AI 生成代码的工程手段 12
2.5.1 可测试性优先的接口设计 13
2.5.2 并发写入的所有权标记 13
2.5.3 人类保留评审与裁决权 13
第三章　意图控制策略 13
3.1 问题的提法方式 14
3.2 上下文管理 14

3.2.1 会话输入契约 15
3.3 约束 AI 的具体手法 16
3.4 引导 AI 修复缺陷 16
第四章 可复现性论证 17
4.1 从零复现 17
4.1.1 依赖完整且固定 18
4.1.2 环境变量最小化 18
4.1.3 三条独立验证路径 19
4.2 演进链路 19
4.3 提交历史与对话快照的对应关系 19
4.4 可验证的边界 21
第五章 工程质量 22
5.1 实测数据 22
5.1.1 分发方式对照：链接 vs 逐份复制 22
5.1.2 跨端同步耗时 23
5.2 测试体系 23
5.3 缺陷修复记录 24
5.4 交付形态 25
第六章 结论 26

提示：本目录为域代码生成，在 Word 中打开时若提示更新域，可选择「是」以获得与正文一致的页码。

第一章　项目背景与问题定义



图 1-1 SkillVault 总览页：技能在 vault 与各 Agent 目录之间的链接状态矩阵

1.1 真实痛点

SkillVault 解决的是一个具体且高频的开发者痛点：当一台机器上同时安装了多个 AI Agent Harness 时，每个 Harness 各自维护一份独立的技能目录，同一个技能被复制成多份副本。用户修改其中一个副本时，其余副本静默地保持旧版本；换机时需要逐个目录手工拷贝；某个技能被误删后无法判断是否还有别处保留。

在 Windows 11 加 WSL Ubuntu 24.04 的双环境开发场景下，这一问题被进一步放大：Windows 侧的 ~/.claude、~/.codex、~/.zcode 等目录与 WSL 侧的 /root/.claude、/root/.codex 之间没有任何关联，一次跨环境的技能变更需要在两个文件系统之间重复操作，且 Windows 与 WSL 对换行符与文件名编码的处理不同，容易在同步过程中产生不可见的字符损坏。

本项目的目标是把散落在各 Harness 目录中的技能收拢到一个本地 Git 仓库中，使其成为唯一真相源，各 Harness 目录通过链接引用该仓库。Windows 与 WSL 两侧各自克隆同一仓库、各自提交同步，全程不依赖任何云端远程仓库。

1.2 核心设计原则

项目在实现过程中确立了五条不可退让的设计原则。这五条原则不是文档性的口号，而是写进了代码结构与测试断言的硬约束。

- 零云远端：vault 目录与本地裸库（bare repository）构成完整闭环。SSH 与 Docker 远程目标仅作为用户显式配置后启用的框架存在，默认关闭且不产生任何外发流量。
- 绝不伪造状态：任何检测（WSL 扫描、Docker 引擎状态、版本查询）不可达时，接口如实返回 stale 或 unknown，取不到的数值返回 null，绝不返回猜测值。
- 密钥安全：API Key 经 Electron safeStorage（Windows DPAPI）加密封存，明文仅经 IPC 通道传递一次，所有展示接口只返回尾 4 位与长度。
- 写操作先备份：配置写入采用「临时文件 → 原子替换 → 回读校验」，需要多文件协同的场景（接口中心档案切换）额外执行「备份 → 写入 → 校验 → 失败补偿恢复」。恢复本身可能受阻，界面会如实标注恢复是否成功，不做无条件回滚的承诺。
- 优雅降级：Docker 引擎未运行、WSL 发行版已停止都是常态，界面显示说明横幅而非报错崩溃。WSL 资源监控页只对 Running 状态的发行版读取指标，绝不为了取数而启动已停止的发行版（见 2.4.2 节的边界说明）。

第二章　架构设计

2.1 总体架构

应用采用 Electron 主进程 + React 渲染进程的经典桌面架构，全部业务编排集中在主进程，渲染进程只负责展示与交互。渲染进程不直接触碰文件系统与外部命令，所有操作经由 preload 暴露的受限 IPC 通道下沉到主进程。

架构的核心特征是「双端各自扫描、共享同一份 Git 仓库」的部署形态。Windows 侧的应用进程与 WSL 侧的伴生 CLI 各自独立地遍历本机文件系统——文件枚举与链接判定始终发生在本侧，不会驱动对方文件系统；扫描得到的元数据（技能清单、Agent 清单、链接状态）以 JSON 形式回传到主进程汇总展示，而技能资产本身只经由 Git 仓库交换。

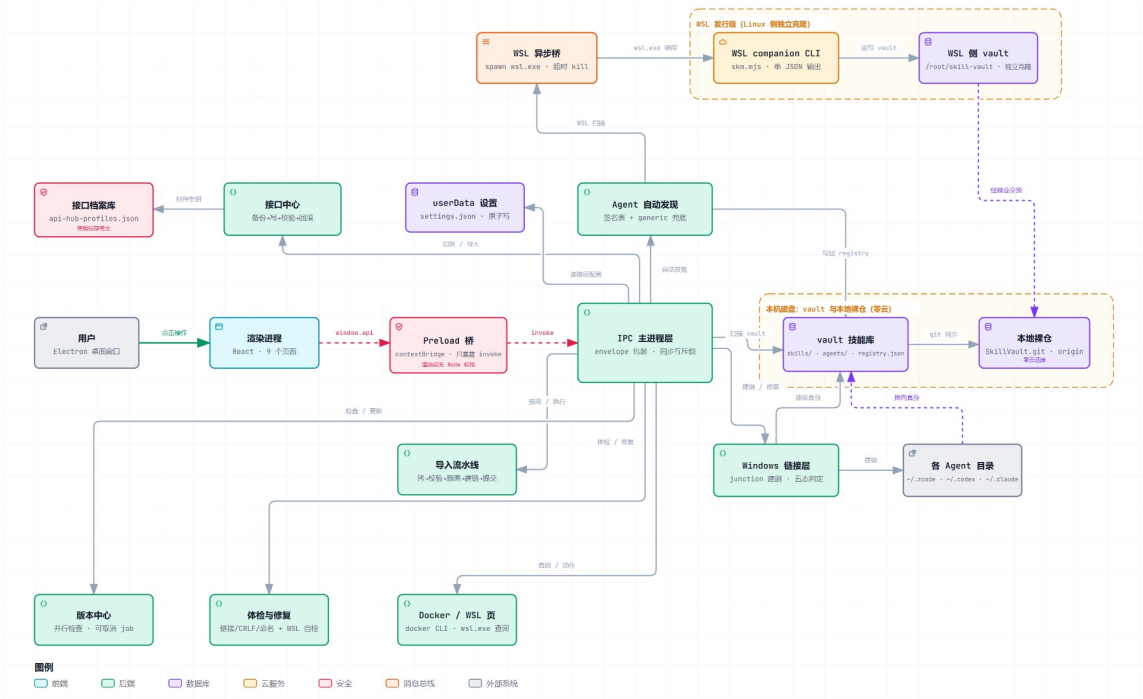


图 2-1 SkillVault 跨 Windows / WSL 技能库体系总体架构

2.1.1 分层视角与组件视角

图 2-1 是从「资产流向」角度描绘的系统全景；图 2-2 则从「交互分层」角度给出同一系统的另一面。两者描述的是同一套实现，但回答的问题不同：前者回答「技能与配置在机器上如何流动」，后者回答「一个用户操作经过哪些代码层」。

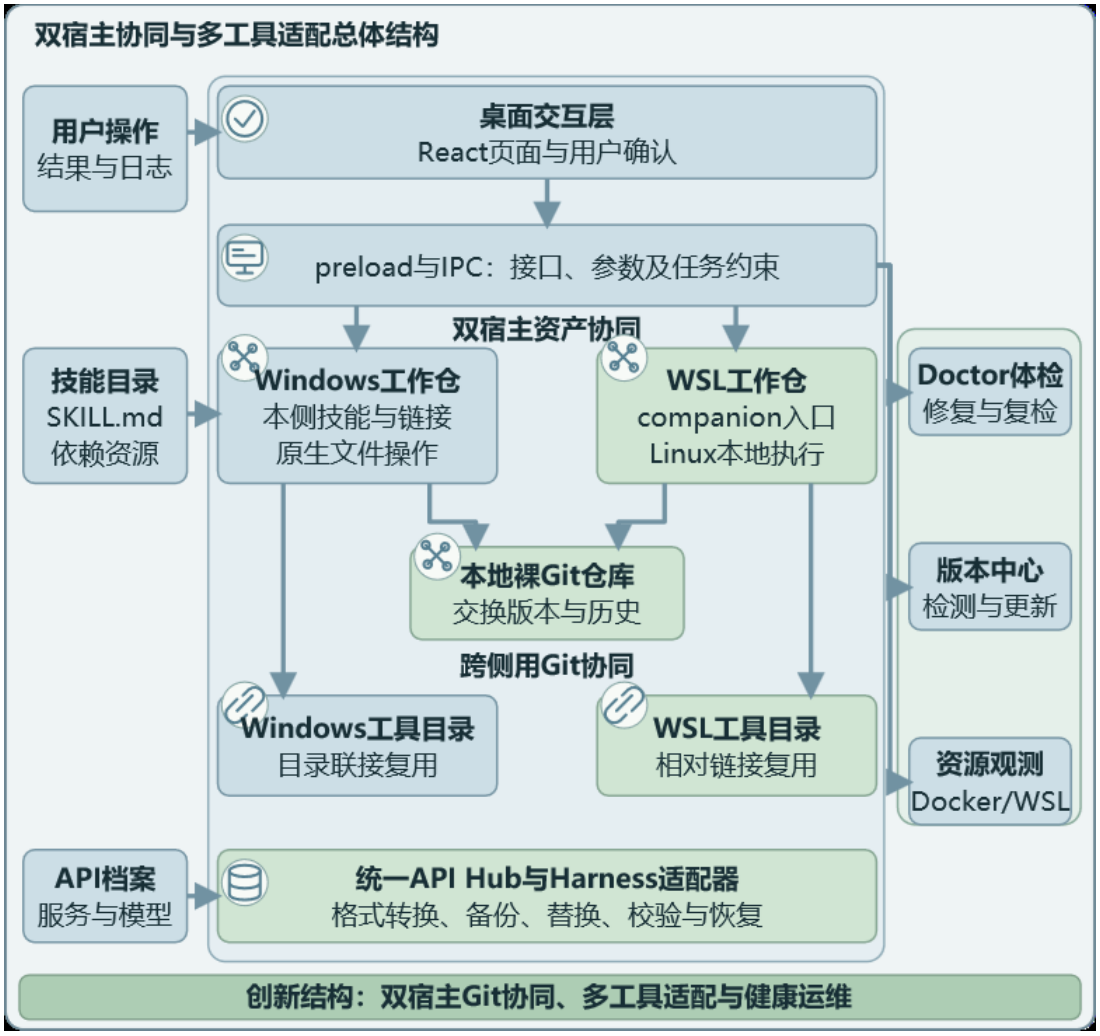


图 2-2 桌面交互层、主进程层、双宿主资产协同与配置适配的总体结构

2.1.2 双宿主协同机制

图 2-3 进一步说明同一份技能资产在两个宿主之间如何流动。核心机制是三件事：Windows 与 WSL 两侧各自保留原生工作仓（各自的文件系统规则、权限模型与工具读取条件都按本侧执行），两侧通过同一个本地裸 Git 仓交换提交，同侧多个 Harness 通过文件链接复用同一份资产。

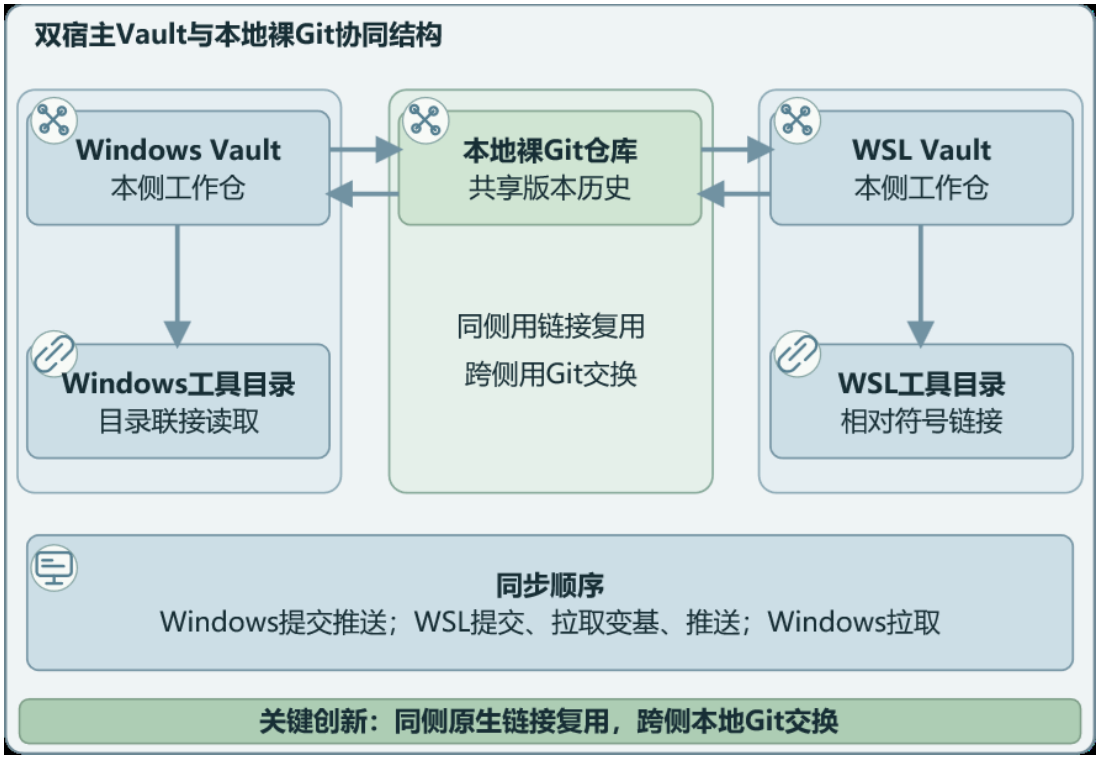


图 2-3 双宿主工作仓、本地裸 Git 仓与同侧链接的协同关系

同步按固定顺序执行：Windows 侧暂存并按需提交后推送到裸仓；WSL 伴生程序提交本侧变更、拉取变基后推送；最后 Windows 侧拉取合并。主进程用串行锁约束相关任务，日志记录每条命令、执行侧与结果。冲突发生时，Windows 合并冲突分支会中止本次合并，用户依据文件与 Git 信息处理后重新同步。

这一顺序设计与「双端各自扫描」的原则互为因果：若两侧互相驱动对方的文件系统操作，跨 /mnt 的大量小文件操作会成为性能瓶颈且难以保证一致。因此文件遍历始终在本侧执行，跨侧只交换 Git 提交与扫描元数据。

2.2 分层结构与模块职责

源码规模为 59 个 TypeScript / TSX 文件、13,673 行生产代码，测试代码 8,124 行 37 个测试文件、567 个测试用例、1,675 条断言，全部通过。生产代码与测试代码的行数比约为 1.7 : 1，这个比例反映了一个取向：核心逻辑的可测性优先于实现的简洁性。

层次	路径	职责	规模
主进程	src/main	全部业务编排：扫描、导入、同步、体	38 文件 / 8,054 行

		检、接口中心、版本中心	
接口中心	src/main/apihub	多 Harness 的 API 档案管理，备份→写→校验→回滚	5 文件 / 1,947 行
版本中心	src/main/versionCenter	8 个 Harness 条目的版本检测与一键更新，5 种更新通道	11 文件 / 1,895 行
渲染进程	src/renderer	React 实现的 9 个页面，无 UI 库、无路由库	12 文件 / 3,809 行
预加载桥	src/preload	contextBridge 白名单暴露 invoke 通道，返回统一 Result 封装	2 文件 / 235 行
共享层	src/shared	三端共用的类型、注册表解析、SKILL.md frontmatter 解析、路径常量	6 文件 / 1,093 行
伴生 CLI	src/companion	WSL 侧零依赖单文件 CLI，8 个子命令，始终输出单个 JSON	1 文件 / 464 行

表 2-1 各层次模块职责与规模

2.2.1 接口层与安全边界

渲染进程通过 preload 暴露的固定方法列表调用主进程，共注册 64 个 IPC handler，按业务域分为 15 类，其中接口中心 11 个、Docker 6 个、其余为单例。preload 桥只暴露 invoke 调用，不暴露 Node 能力，配合主进程 contextIsolation 开启构成渲染层与系统层之间的单向边界。

所有 IPC 返回值统一使用 Result<T> 封装，形如 { ok: true, data } 或 { ok: false, error }，error 为可读的中文描述字符串而非错误码枚举。渲染层据此渲染状态，不需要解析异常堆栈；若需按错误类型分支处理，目前依赖字符串匹配，属于已知的后续改进项。

2.2.2 共享层的单一真相源设计

src/shared 是主进程、渲染进程与 WSL 伴生 CLI 三方共用的代码层，集中定义了 registry.json 的版本化解析逻辑、SKILL.md 的 frontmatter 解析规则、Agent 签名的识别算法与跨端路径常量。

这一层的存在直接决定了「Windows 与 WSL 两侧对同一份 registry.json 的理解必须完全一致」这一约束能否成立。如果两侧各写一套解析器，registry 的 schema 一旦演进就会出现静默分歧：Windows 侧认为某个 agent 是 builtin 且 enabled，WSL 侧可能因为不认识新的 source 字段而把它当成未知条目跳过，从而导致两侧扫描结果不一致，链接状态判定出现偏差。将解析逻辑收敛到共享层是这个约束的唯一可靠实现方式。

registry.json 当前为 version 3 格式，在 version 1 与 version 2 基础上增加了 label、source（builtin / discovered / manual）、enabled、sigId、status（active / missing）字段读取端对 v1、v2 均保持兼容。

2.2.3 Agent 自适应发现机制

固定适配有限的 Harness 列表无法覆盖真实开发环境——用户在半年内完全可能换用新的 Agent 框架。为此项目实现了签名驱动的自动发现机制，在 src/shared/agentSignatures.ts 中定义了 16 个 Harness 签名：claude、codex、zcode、kimi、dsh、grok、gemini、qwen、opencode、kilo、pi、crush、goose、copilot、cursor，以及通用的 agents-shared 目录约定。

每个签名通过候选配置目录的存在性识别一个 Harness。这里需要区分三个状态签名命中（候选配置目录存在）、技能目录存在（其下 skills/ 实际存在）、工具可运行（该 CLI 是否安装并可执行）——本项目的自动发现只覆盖前两个状态，第三项需由用户自行确认。因此「已发现」表示该框架的目录结构符合约定，不等于该工具已安装或可正常使用。若所有签名均未命中，则由 genericSweep 兜底机制扫描用户主目录下任意形如 .<name>/skills 的目录，并要求其中至少一个技能子目录含 SKILL.md，否则不视为一个有效技能目录。兜底机制会跳过已被签名覆盖的目录，避免同一 Harness 被重复上报。

发现结果通过 mergeDiscovered 幂等地合入注册表：已存在的条目按 sigId 匹配更新，不存在的条目追加，标记为 missing 的条目保留而不删除——因为 Harness 可能只是暂时未启动，删除条目会导致用户的启停配置丢失。

这套机制带来一个直接的产品后果：接口中心与版本中心的条目不再是硬编码列表，而是由注册表驱动动态合成。注册表中检测到但接口中心尚未实现适配的

Harness，会渲染成一张说明卡而非空白页，告知用户该框架已被识别但暂不支持档案切换。

2.2.4 源码索引

各设计对象与源码位置的对应关系如下，评审可据此逐项核对实现。

设计对象	主要源码位置	核对要点
双宿主协同	src/main/sync.ts、src/companion/skm.ts	双端提交与拉取顺序、同步日志、冲突处理
跨进程通信	src/main/ipc.ts、src/preload/index.ts	页面调用入口、参数校验、同步串行控制
技能导入与链接	src/main/importer.ts、src/main/winLinks.ts	导入检查、复制校验、链接建立与五态识别
API 档案管理	src/main/apihub/index.ts、transforms.ts、store.ts	字段转换、备份、写入、校验与补偿恢复
健康检查与修复	src/main/doctor.ts	问题分级、可修复项识别、定向修复
WSL 扫描与降级	src/main/wslScan.ts、src/main/wslBridge.ts	伴生程序调用、超时处理、缓存时间与失败原因
Agent 自动发现	src/main/agentDiscover.ts、src/shared/agentSignatures.ts	16 个签名、generic 兜底、幂等合并
版本中心	src/main/versionCenter/	多通道检测、更新任务、增量日志、取消与失败隔离
Docker / WSL 状态	src/main/docker/index.ts、src/main/wslmon/index.ts	资源快照、缺测处理、停止发行版保护
页面与共享类型	src/renderer/src/pages/、src/shared/	功能页面、状态展示、注册信息与跨端类型

表 2-2 软件设计与源码索引

2.3 关键机制：目录链接与链接状态判定

Windows 侧的链接层需要判定每个 Agent 技能目录与 vault 之间的真实关系，这一判定结果直接驱动仪表盘的状态展示与一键修复的正确性。

判定共分五态：linked（符号链接指向 vault 内正确目标）、wrong-target（是指链接但目标错误）、real-dir（是真实目录而非链接）、missing（目录不存在）、vault-missing（vault 本身缺失）。

一个容易被忽略的例外是硬链接。ZCode 的 .zcode/agents 目录被有意设计为真实目录而非符号链接，因为实测发现 ZCode 在重启时会清理符号链接目录。项目的解决方案是：识别同设备同 inode 的硬链接共享，将其判定为 linked 并标注「硬链接共享」，并提供一键修复功能重建硬链接，把不一致的文件移入回收目录而非直接删除——该目录默认位于 vault 的同级，形如 <vault 父目录>/.trash-<时间戳>，修复前可从该目录取回原文件。

2.4 安全模型

密钥在本应用自己的档案库中遵循「一次明文入内存、落盘即加密」原则。用户填写 API Key 时，明文经 IPC 传入主进程，立即经 Electron safeStorage（Windows 上为 DPAPI）加密后 base64 落盘，渲染层此后只能读到脱敏视图（尾 4 位加长度）。当 safeStorage API 调用失败时（例如应用 ready 之前调用、或系统加密服务不可用），降级为 base64 明文存储，并在数据上打 plainStore 标记，界面据此显示降级状态提示。

必须明确两点边界：其一，base64 不是加密，只是编码——降级文件一旦泄露即可直接还原出明文，因此该路径下不存在需要额外保管的解密密钥；其二，本应用的加密只覆盖自己的档案库——执行档案切换时，密钥会被解密并写入目标 Harness 的配置文件与备份文件，那些文件由目标工具自身管理，不受本应用的加密保护。因此目标配置与备份应按敏感文件对待，限制访问权限；DPAPI 加密的档案则受当前用户凭据限制，无法直接迁移到另一台机器或另一个用户账户。

配置写入路径统一实现于 src/main/apihub/store.ts 与 src/main/settings.ts，写入临时文件后原子 rename 覆盖目标文件，读取路径对 NUL 字节与损坏 JSON 做显式处理。



图 2-4 接口中心：档案库与密钥脱敏视图（密钥仅显示尾 4 位与长度）

2.4.1 配置写入与补偿恢复的边界

配置切换的可靠性设计是本项目安全模型中最需要精确表述的部分。图 2-5 给出单文件原子替换与多文件补偿恢复两条路径的分工。

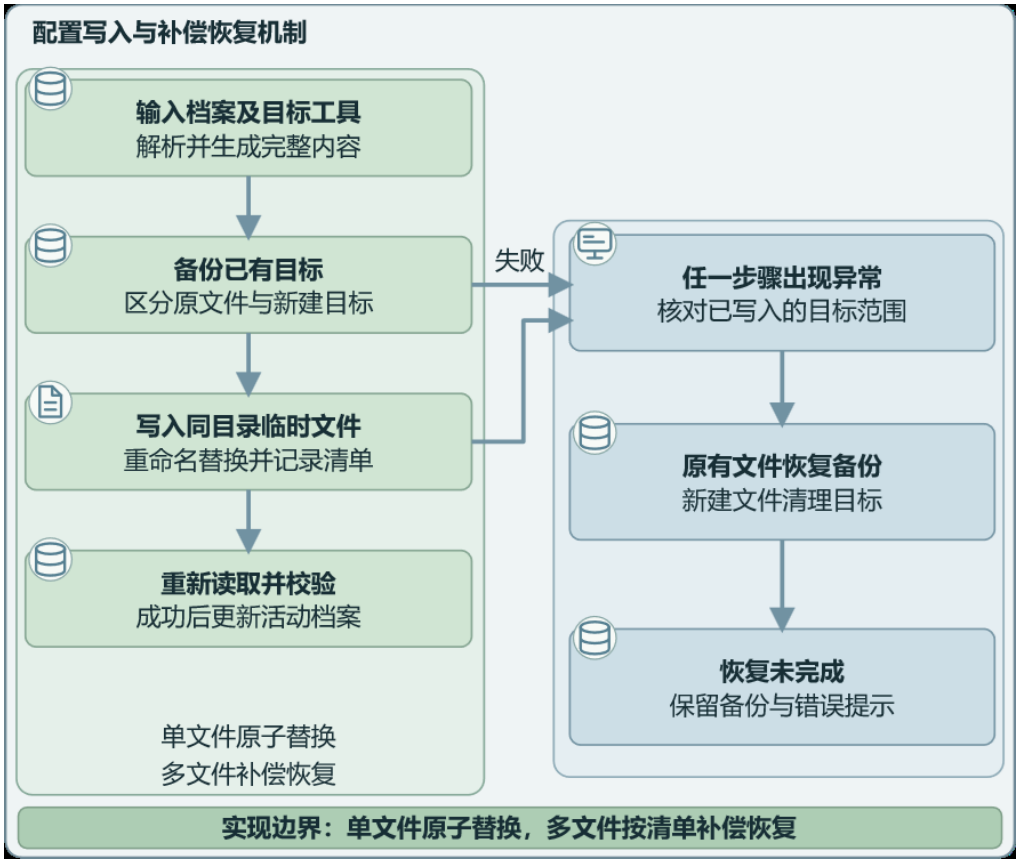


图 2-5 配置写入与补偿恢复机制的实现路径与异常分支

单文件替换具有原子性：内容完整写入同目录临时文件后以 rename 替换目标，因此不存在写到一半的中间状态。多文件场景则依赖补偿操作——系统按「目标路径—备份路径」记录恢复对象并跟踪已完成替换的目标，中途失败时，已存在的文件按备份恢复，本次新建的目标按清单清理。

这一机制存在明确的实现边界，必须如实声明：恢复过程中发生的异常会被捕获，但返回信息主要保留原始切换错误，因此失败后的可靠性判断还需核对目标文件与备份内容；活动档案的登记状态也可能与实际配置文件不一致。文件被其他进程占用、权限不足或存在外部并发写入时，恢复动作本身也可能受阻。

还需要限定「原子」的适用范围：单文件原子替换指的是「内容完整写入同目录临时文件后以 rename 替换目标」，因此不存在写到一半的中间状态——但它不构成多文件事务，不保证掉电持久性，也不保证在外部并发写入下安全。多文件场景依赖补偿恢复而非事务回滚。单文件原子替换与多文件补偿恢复应作为两个独立性质分别验收，不可互相替代。

2.4.2 降级路径的信息保真

健康运维与资源查询遵循同一条原则：查询失败不等于资源为空，缓存结果不等于当前状态。系统将扫描结果分为实时结果、带时间戳的过期缓存与不可达三种状态；资源读取失败时保留已解析指标，将缺测字段置为空值，并明确标注缓存时间与失败原因。图 2-6 给出健康维护流程与资源查询的降级分支。

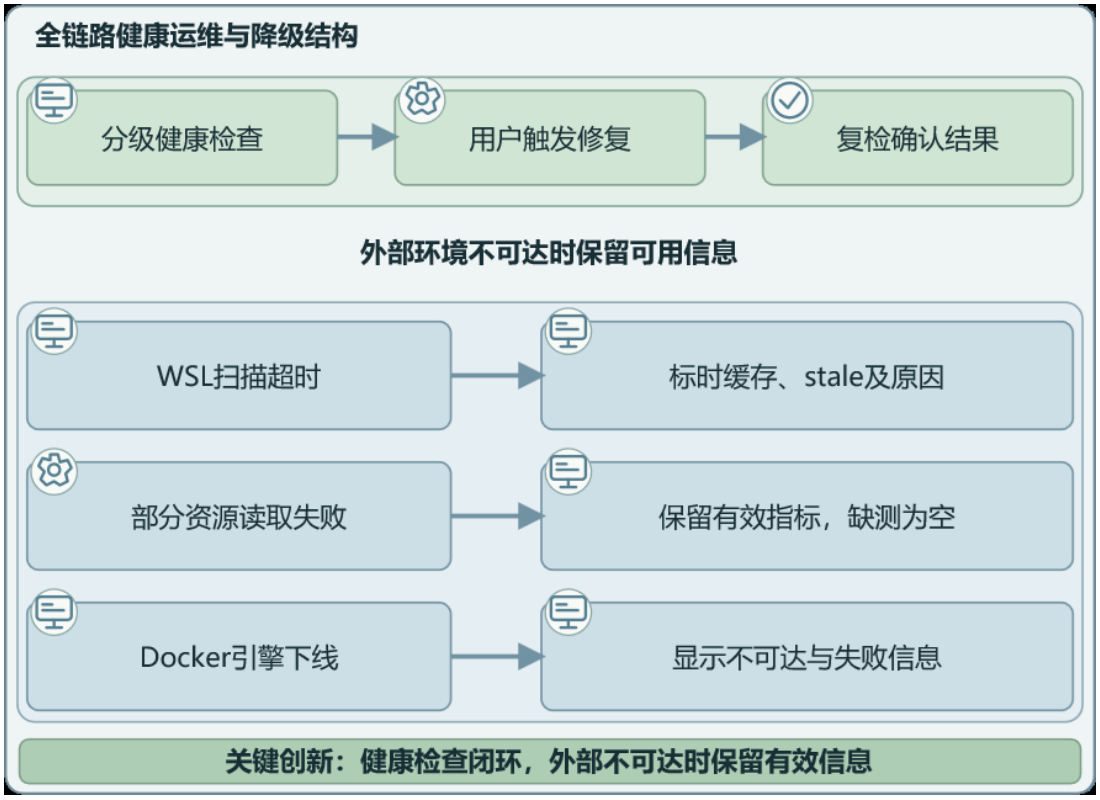


图 2-6 健康维护流程与资源查询的降级分支

这一设计的作用是让用户能区分三种不同的情况：过去的结果、当前读取失败、以及实际零值。例如 WSL 扫描失败时，一份过期缓存仍能帮助用户查看曾经识别过的技能，但不能代表当前目录已验证通过，因此页面必须同时显示缓存时间与失败原因。Docker 统计部分缺失时，已取得的容器指标仍有参考价值，缺测字段保持为空值，避免被误读为实际使用量为零。

2.5 约束 AI 生成代码的工程手段

本项目的业务代码由 AI 在人的评审下生成。约束手段不是靠提示词里的「请写好代码」，而是靠架构与流程上可验证的机制。

为避免过度声明，需要说明代码的来源构成：主体业务逻辑（主进程、渲染进程、共享层、伴生 CLI）由 AI 生成；此外包含少量人工直接编辑的部分——项目脚

手架与构建配置、图标与打包资源、以及经人工复核后改写的测试文件。四条 Prompt 链快照记录的是生成过程中的代表性往返，并非对每一行代码来源的穷尽证明。

2.5.1 可测试性优先的接口设计

业务逻辑与 I/O 分离是核心约束。核心模块（导入、链接、同步、体检、接口中心、版本中心）把文件系统、home 目录、时钟、加密器、子进程启动器作为依赖注入传入，测试在系统临时目录中演练，真实文件零改动；外部命令（docker、wsl、git、npm）经 fakeSpawn 模拟。少数模块仍直接使用模块级 fs，其影响被临时目录隔离兜住。这使得 567 个测试用例在 Windows 11 + Node 24 环境下可稳定运行且不污染用户环境。

2.5.2 并发写入的所有权标记

当多个 Agent 同时向同一文件追加代码时，冲突不可避免。项目在 src/main/ipc.ts、src/preload/index.ts、src/shared/types.ts 三个文件中预插入区域所有权标记：

```
// ===== REGION-IPC-APIHUB（并发子代理在下方追加 ipcMain.handle，勿改本行及以上内容） =====
```

每个并行工作的 Agent 只在自己的标记行下方追加内容，互不重叠。在 v11 轮次的两个并发 Agent 交付中，这一机制实现了零冲突合流——两个 Agent 分别完成了接口中心的动态化改造与导入自动扫描功能，合并时未产生任何冲突标记或人工介入。

2.5.3 人类保留评审与裁决权

架构决策、需求定义与最终验收始终由人负责，AI 承担实现与自检。安全扫描器在多次交付中对生产代码报出路径穿越类告警，这些告警经人工逐条核对代码证据后裁定为误报，并记录在案以避免后续会话反复尝试改写已验证正确的实现。反过来，测试侧的真实问题则按扫描器提示改写通过——行为等价的改写共涉及 8 个测试文件。

第三章　意图控制策略

3.1 问题的提法方式

Vibe Coding 的成败往往不取决于模型能力，而取决于提问方式。本项目的需求提法遵循一条原则：先描述痛点与约束，再描述期望行为，最后才描述实现形式。

典型的对照可以看第 1.1 节所述痛点在整个项目周期中的三次表述。第一次表述是场景化的自然语言描述，包含了环境约束（Windows 与 WSL 双环境）、概念锚定（中央仓库、唯一真相源）与行为预期（各目录链接过去），但没有任何技术词汇。第二次表述追加了一条工程约束：「wsl 跨 /mnt 做大量文件操作会崩，应该 wsl 自己扫自己，windows 自己扫 Windows」。第三次表述追加了部署形态约束：「vault 仓库不应该放 f 盘，我是说源码放 f 盘而已，仓库放 c 盘吧」。

这三次追加都直接改变了架构决策。第一次追加排除了「Windows 侧跨 /mnt 驱动 WSL 扫描」的方案，确立了双端各自扫描的架构；第二次追加改变了数据存放位置与源码分离的原则。对照第 2.1 节的图 2-2 可以确认：图中的 WSL 伴生 CLI 组件正是第一次追加约束的产物，而 vault 位于本机磁盘、零云远端的形态则来自第二次追加。

这条经验在项目后期被固化为工作方式：面对新需求，先完成只读侦察并输出书面方案与选项，待人确认后再进入实现阶段，人保留架构决策权与最终验收权。

3.2 上下文管理

长周期项目的上下文衰减是 Vibe Coding 的主要失败模式之一。SkillVault 的完整开发周期跨越 2026 年 9 月 1 日至 10 月 7 日，经历了 12 轮功能增量与至少 5 轮全量重审。

控制上下文衰减依赖三件事。第一是注册表与共享层承担的项目记忆——registry.json 记录了每个 Agent 的平台、skillsDir、include 规则与可选 agentsDir，agentSignatures.ts 记录了全部识别规则，新会话读取这两个文件即可恢复「Agent 注册与发现」相关的上下文，无需重读全部源码；涉及同步、写入恢复、安全边界与界面状态的细节仍需查阅对应模块。

第二是分层的信息边界：设计文档只描述结构与约束，实现细节留在代码注释中，避免在长上下文中重复携带已经固化为代码的细节。第三是提交历史作为增量

记忆载体——每次功能落地形成一个语义完整的提交，提交信息本身记录了该轮的能力增量与测试数变化。

一个具体例子是 registry 版本演进的处理方式。v3 引入的 source、sigId、enabled 字段改变了整个注册表的语义，而 v1 与 v2 的存量数据仍在用户机器上。实现方案是让读取端对三个版本都保持兼容，同时为 mergeDiscovered 提供幂等保证，使「发现 → 合入 → 再次发现」不产生漂移。测试方面新增 registry-v3.test.ts 共 252 行，覆盖合并的幂等性、missing 条目不被删除、停用条目在扫描与链接操作中被一致拒绝等边界。

3.2.1 会话输入契约

上下文管理不能只靠「记得什么」，还必须规定每轮交给模型的材料与边界。本项目在每轮实现开始前遵循下述输入契约，它既是给 AI 的上下文说明，也是给人的控制点。

契约项	内容	作用
基线状态	当前提交、工作区改动、既有测试与类型检查结果	让 AI 从实际状态出发，而不是从想象出发
已确认需求与非目标	本轮要做什么、明确不做什么	防止范围蔓延（如「只加功能，不要顺手重构」）
允许改动的文件	具体文件路径或目录	把写入面收窄，冲突面随之收窄
不得改变的接口与数据格式	已发布的函数签名、配置文件格式、注册表结构	避免单轮改动破坏外部兼容
必须执行的验证命令	pnpm test / pnpm typecheck / 特定单测	把「完成」定义成可执行的判据
证据不足时的停止条件	遇到不明确的边界先问，不自行假设	避免 AI 用猜测填补信息缺口

表 3-2 会话输入契约的六个组成项

需要如实说明这套契约的强度：它是协作约定，不是权限隔离。区域所有权标记（如 REGION- 前缀的注释行）依靠执行方遵守，不会自动阻止越界修改，也不天然保证两个并行 Agent 的改动在语义上兼容。真正的约束来自两道客观防线——允

许改动的文件清单由人划定，以及每轮结束后由独立于交付者的复核者重新运行验证命令并核对实际产物。

3.3 约束 AI 的具体手法

项目使用的约束手段按强度递增排列，形成一个从软到硬的阶梯。

层级	手段	实例	可验证性
提示层	需求中显式声明禁止事项	用户明确「不要用现成队列模块」式的具体禁令	低，靠人审
提示层	指定技术栈与拒绝方案	拒绝 Python 伴生 CLI，确定 Node/TS 全栈	低，靠人审
接口层	规定方法签名与参数语义	skillOpen 三重校验：名称正则、路径包含、扫描成员	高，测试覆盖
架构层	区域标记划定写入边界	REGION- 标记行下的追加式协作	高，冲突可检测
架构层	依赖注入隔离外部依赖	文件系统、时钟、加密器、子进程全部注入	高，测试可演练
流程层	人类保留架构与验收权	侦察→书面方案→确认→实现→独立于交付者的复核	中，靠流程保障
流程层	AI 自述不作为验收依据	每个 Agent 的完成报告均被独立命令复核	高，客观证据

表 3-1 意图控制手段的可验证性分层

最值得强调的是最后一行。每一轮子 Agent 交付后，验收方都不采信子 Agent 的自述结论，而是用自己的命令重跑测试、检查链接状态、核对 git 提交，逐条复验这一做法在多个轮次中实际拦截了子 Agent 报告中的偏差——包括自述已修复但实际未修复的条目、以及自述测试通过但实际断言数量不符的情况。

3.4 引导 AI 修复缺陷

缺陷修复环节的 Prompt 质量直接决定修复的彻底程度。项目在多个轮次中采用的做法是：先让 AI 复述现象与自身判断，再要求给出证据，最后才允许改动代码。

以 2026 年 9 月 7 日的全量重审为例。该轮由一句「重审项目，解决所有 bug」触发。AI 的首个动作不是改代码，而是建立基线——先运行测试确认 404 个用例全部通过，再运行双 tsconfig 类型检查确认无错误。两个门禁全绿之后才进入真正的缺陷审查，此时的判断依据从「自动化没报错」转为「自动化没覆盖到的逻辑缺陷」。

随后派生 6 个并行审查 Agent，分别负责不同模块的深度审查，同时主流程自行阅读核心文件。审查 Agent 回报了约 45 个疑点，主流程逐条核对代码证据后确认其中约 40 个为真实缺陷并完成修复，覆盖四大类：数据损坏类（如 CRLF 处理在重编码时把 GBK 与 UTF-16 文本文件永久写坏）、并发与状态类（如同步操作可重入导致锁失效、jobs 状态被 cancelled 后的 done 覆盖）、边界判定类（如 Windows 保留名 con/nul/com1-9 未被拒绝、CRLF 修复误用 add -A 提交了无关草稿）、以及安全类（如修复载荷仅由文件名派生导致的路径穿越写风险）。疑点数、确认缺陷数与跨轮次累计数口径不同，本文档后续均采用「确认缺陷数」表述。

这一轮同时产出了 22 个新的回归测试用例，总测试数从 404 增至 426。值得注意的是新增测试的选择标准：它们针对的是修复过程中暴露出的「修复本身可能引入的新错误」，而非简单复现原缺陷。典型例子是 apihub 备份数组的索引错位——原实现用数组下标对应备份文件，当第一个文件缺失时会抛出 TypeError 而掩盖真正的原始错误；修复改用 Map 后，又补了一条多文件写入中途失败导致文件对错配的测试。

修复过程本身产出了两套可复用资产。其一是代码注释中的「为什么」记录：多处实现带有说明为何选择某种写法以及顺序错误会导致什么后果的注释，这些注释在后续维护中承担了知识传递功能。其二是本项目自身的技术文档与架构图——它们产自同一批源码事实，因此与代码保持同步。

第四章　可复现性论证

4.1 从零复现

复现路径的完整说明见仓库根目录的 REPRODUCE.md，本文档与该文件、docs/ 下的实测数据与 Prompt 链快照同属源码锚点 2e3ede7，评审可按该提交逐项核对。本节给出判断可复现性的三个维度。

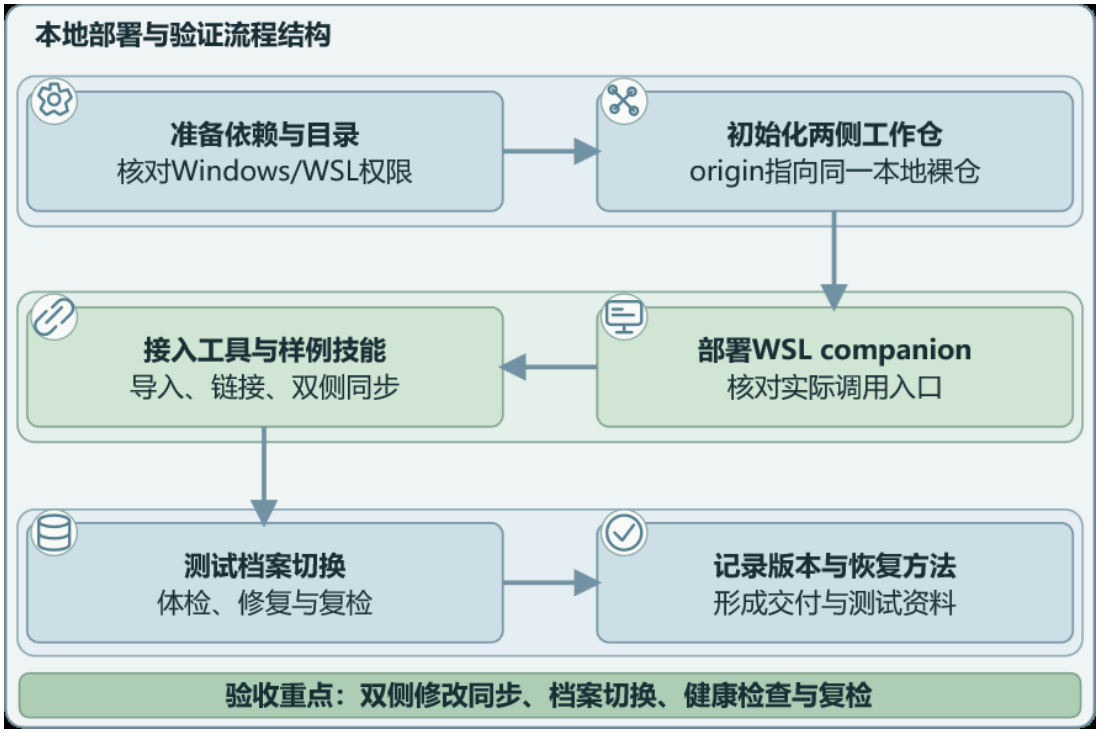


图 4-1 本地部署、样例验收与交付顺序

4.1.1 依赖完整且固定

项目使用 pnpm 管理依赖，依赖树由 pnpm-lock.yaml 锁定（154,452 字节）。运行时依赖仅 2 个（react 与 react-dom），其余全部为构建期依赖；Electron 本身由 electron-builder 一并打包进成品，因此目标机器无需安装 Node。Electron 与 electron-builder 的二进制下载源在 .npmrc 中配置为国内镜像，适配网络受限环境。复现时建议使用 pnpm 的冻结锁文件安装（pnpm install --frozen-lockfile），并记录 Node 与 pnpm 的实际版本。

4.1.2 环境变量最小化

用户需要配置的业务环境变量只有一个：SKM_VAULT，供 WSL 伴生 CLI 指定 vault 路径，默认值为 /root/skill-vault。Windows 侧的默认路径由 os.homedir() 派生而非硬编码——这一改动源于跨机故障：原先硬编码的 C:\Users\sakuya 路径在换机后立即失效。模板见仓库根目录 .env.example。

工具内部另有一组环境变量，不属于用户配置范围：ELECTRON_RENDERER_URL 仅在开发模式用于指向渲染层调试服务器，WSL_UTF8=1 由程序在调用 wsl.exe 时自行注入以保证输出编码。

4.1.3 三条独立验证路径

复现者可以从三个方向独立验证系统行为。

验证路径	命令	验证内容	预期结果
单元测试	pnpm test	37 个测试文件、567 个用例	全部通过
类型检查	pnpm typecheck	主进程与渲染进程双 tsconfig	零错误
伴生自检	node skm.mjs selfcheck	WSL 侧身份、origin 指向与执行权限	输出单个 JSON，需逐字段核对

表 4-1 三条独立验证路径

表 4-1 给出的是可机械执行的三条路径，但需要强调：它们各自验证的是不同的事实——「测试通过」证明的是被测逻辑符合断言，「窗口打开」证明的是应用可启动，「双端同步正确」证明的是跨宿主协作无误，「开发过程可追溯」证明的是演进链完整。这四件事不能相互替代，本文档分别在第 5.2 节、第 2.1 节、第 5.1 节与第 4.3 节给出对应证据。selfcheck 的顶层 ok 字段同样不能替代字段级判定，需按其输出的身份、originOk、权限项逐项核对。

4.2 演进链路

可复现性不仅指最终产物可重建，还要求开发过程可被追溯。本项目的演进过程在两处留下了结构化记录：Git 提交历史与 AI 对话历史快照。

Git 提交历史记录了从初始实现到 v11 版本的能力增量。每一轮功能落地对应一组语义完整的提交，提交信息包含本轮的能力增量与测试数变化。

AI 对话历史快照记录了需求提出、方案讨论、缺陷定位与修复引导的完整往返。快照包含至少三条核心迭代的完整 Prompt 链，每条链记录人的提问、AI 的分析建议与修改方案，以及人如何引导 AI 定位并修复具体缺陷。

4.3 提交历史与对话快照的对应关系

提交历史与对话快照不是两份独立材料，而是同一演进过程的两个投影。二者之间是主题级关联，不是严格一一对应：

演进阶段	整理后的提交（语义化）	变更规模	对应 Prompt 链快照

应用骨架与链接分发	feat(core): Electron 应用骨架、共享层与 vault 链接分发	36 文件 / 9,130 行	docs/prompt-chains/01-从零构建跨端技能库.md
版本中心	feat(version-center): 多 Harness 版本检测与一键更新	15 文件 / 3,432 行	（含于链一）
接口中心	feat(apihub): 多 Harness API 档案切换	14 文件 / 4,998 行	（含于链一）
跨机乱码修复	fix(encoding): 跨机乱码全链路修复	5 文件 / 339 行	docs/prompt-chains/03-跨机乱码全链路排查.md
Agent 自适应	feat(agent-discovery): Agent 自适应发现与 registry v3	9 文件 / 2,244 行	docs/prompt-chains/04-Agent 自适应发现.md
缺陷修复	fix: 多轮缺陷修复（并入各功能提交）	22 个新增回归用例	docs/prompt-chains/02-全量重审修复 40 个缺陷.md

表 4-2 提交历史与 Prompt 链快照的主题级关联

必须区分两类历史：仓库当前的 17 个提交是对最终状态按模块所做的**事后整理**，目的是让评审能按功能单位阅读代码，它不等同于原始开发时序——原始的提交先后顺序与中间态并未保留，语义化提交之间的日期也是按轮次归属而非实际推送时间标注。因此表 4-2 是主题级的多对多关联，不应被理解为「某一提交由某一条 Prompt 链直接产生」的一一对应证明。真正的过程证据由 docs/prompt-chains/ 下的会话导出承担。

仓库的完整提交历史经重构为 17 个语义化提交（截至源码锚点 2e3ede7），每个提交对应一个功能模块的落地，而非一次性导入全部功能。四条 Prompt 链快照以 Markdown 原文形式保存在 docs/prompt-chains/ 目录下，由本机 AI 会话数据库直接导出，未经人工改写或美化，保留了原始提问、AI 回复、模型标识、工具调用次数与时间戳，评审者可据此复核开发过程的真实性。

Prompt 链快照	**人的输入**	**系统消息**	**AI 回复**	**工具调用**	**主题**
链一 · 从零构	22	16	266	306	架构意图从痛点陈述到部署形

建					态修正
链二 · 全量重审	2	22	233	256	建立基线 → 6 个并行审查 Agent → 主 Agent 核实
链三 · 乱码排查	4	7	78	100	现象 → 环境差异 → 编码点审计 → 字节级改造
链四 · 自适应发现	9	10	106	121	否定旧方案 → 拆解子需求 → 落地签名驱动

表 4-3 四条 Prompt 链快照的消息来源构成

计数口径：会话库中同一条人工输入可能重复落库，表内「人的输入」已按内容去重；会话框架注入的消息（工具使用提醒、会话续接提示等）单列为「系统消息」，不计入提问编号。四条链合计 37 条真实人工输入、783 次工具调用。链二的真实人工输入只有 2 条，但 AI 侧展开了 233 次回复与 256 次工具调用——该轮的价值在于「一次指令 → 六个并行审查 Agent → 主 Agent 逐条核实发现」的调度过程，而非提问数量。

4.4 可验证的边界

为避免过度承诺，本节明确列出当前版本尚未覆盖的范围。

- SSH 与 Docker 远程目标目前为框架状态：已实现目标管理、探测与 git-bundle 传输抽象，并有基于 fake 的单元测试覆盖，但未接入真实远程目标，因此远程同步的实际链路未经过真实验证。
- Docker 与 WSL 页面的操作仅在开发机 Docker Desktop 未运行的环境下验证了降级路径，容器生命周期操作的完整链路需在 Docker 引擎运行的环境中复验。
- 部分旧版 WSL 忽略 WSL_UTF8 环境变量并返回 UTF-16 输出时界面仍会出现乱码。微软在 0.64.0 的发布说明中才把 WSL_UTF8 列为新增能力，因此本项目使用的版本阈值应以该版本为参考而非更早的版本；当前实现未做代码层兜底，需升级目标机 WSL 版本解决。
- 第 5.1 节的实测数据在单机单卷条件下取得。跨卷（vault 与 Agent 目录位于不同 NTFS 卷）时硬链接共享不可用，需改用 junction 整目录链接；但对于会清

理 junction 的 Harness，这并不构成有效替代方案——此时应明确拒绝并提示将 vault 迁移到同卷。

- 硬链接共享下的修改传播存在写入方式差异：原地修改共享文件会立即反映到所有副本，而以「临时文件替换原路径」方式保存则会破坏原有硬链接关系，可能使各副本分叉。该行为已实测确认，但尚未对各类编辑器的保存方式做穷尽验证。

第五章　工程质量

5.1 实测数据

自动化测试验证的是功能正确性；本节补充的是「实际有多快、有多省」的可量化对照。全部数据由脚本在同一台机器上实测生成，原始记录见仓库中的 docs/measured-data.json，测试环境为 Windows 11、Node v24.21.0、Git 2.54.0、vault 与 Agent 目录位于同一 NTFS 卷。

5.1.1 分发方式对照：链接 vs 逐份复制

场景为 26 个技能分发到 20 个 Agent 目录，每个技能 3 个文件、源数据合计约 418 KB。两组实验使用完全相同的数据规模与目录结构，唯一差别是建立分发关系的手段——一组调用 Node 的 fs.symlinkSync(..., "junction")，另一组调用 Python 的 shutil.copytree。两者运行在不同的宿主进程与实现语言中，进程启动开销未被单独扣除，因此 11.0 倍这一比值应理解为「端到端观测量」而非严格的算法级对比。原始记录见 docs/measured-data.json。

建立分发关系	耗时	技能文件内容副本	一次改动的传播成本
链接分发（junction）	338.9 ms	0 份（不增加内容副本）	0 次额外写入，20 个目录立即生效
逐份复制（copy）	3,725.9 ms	20 份（本次约 8.3 MB）	需重写 20 份副本
差异	11.0 倍加速	省去 20 份冗余副本	后续每次修改均省去 20 次写入

表 5-1 26 技能 × 20 Agent 的分发方式对照（实测）

单点更新的对比更为直观：修改 vault 中的一个技能文件后，链接分发下 20 个 Agent 目录读取该文件均已立即返回新内容，验证耗时 4.9 ms，额外写入次数为 0；逐份复制方案则需要重写 20 份副本，耗时 41.2 ms。这正是「一处修改、多处生效」的具体含义。

5.1.2 跨端同步耗时

跨端同步按三步执行：Windows 提交推送 → WSL 伴生程序拉取变基后推送 → Windows 拉取合并。以 26 个技能的工作仓为例，首次完整同步耗时 1,944.9 ms，其中 Windows 推送 720.4 ms、WSL 拉取变基推送 915.5 ms、Windows 拉取合并 308.9 ms。

后续的单文件增量同步连续测 5 轮，耗时分别为 1,122.6 / 990.3 / 976.5 / 1,000.5 / 976.0 ms，平均 1,013.2 ms，最小 976.0 ms，最大 1,122.6 ms。波动幅度约 15%，主要来自 Git 每次调用的进程启动开销，而非文件传输——这一点由下一组规模对照数据佐证。

技能数	文件数	首次推送耗时	增量推送耗时
10	30	471.2 ms	353.9 ms
26	78	625.0 ms	363.2 ms
100	300	490.9 ms	413.4 ms
500	1,500	419.4 ms	432.8 ms

表 5-2 不同规模的同步耗时（实测）

表 5-2 的观察是：在本次测试的数据集与计时边界内，技能数量从 10 增长到 500（文件数从 30 增至 1,500，增量约 50 倍）时，推送阶段耗时基本不变，初始推送稳定在 420–625 ms 区间。这是一个待进一步验证的解释而非结论：可能与 Git 的内容寻址存储有关（提交记录的是内容寻址的 blob 与 tree 引用），但也受数据可压缩性、内容重复度与 Git 对象数量影响；且该表测量的仅是推送阶段而非完整三步同步，单点计时也无法分解耗时构成。此处的结论仅限于本机、本数据集与本计时方式，不外推到其他环境。

5.2 测试体系

测试采用 vitest，覆盖 37 个文件、567 个用例、1,675 条断言，全部通过。测试设计遵循三条原则。

第一是隔离原则：核心模块的文件系统、home 目录、时钟、加密器、子进程启动器可注入，测试在系统临时目录中演练，真实用户文件零改动；其余模块的文件操作同样被临时目录隔离。外部命令经 fakeSpawn 模拟，因此测试不依赖 Docker 引擎运行、WSL 发行版启动或网络可达。

第二是回归优先原则：修复缺陷时同步补充测试，且新增测试针对修复过程中暴露的新风险点，而非简单复现原缺陷。2026 年 9 月 7 日一轮的 22 个新增用例即为此模式。

第三是反例覆盖原则：边界与异常路径的测试数量不少于正常路径。典型如 apihub-transforms 的 27 个用例覆盖了 CRLF 链式破坏、备份索引错位、中途写入失败、BOM 处理、保留名拒绝、内嵌符号链接、端口号校验等场景。

测试文件	用例数	覆盖重点
apihub-runtime	28	档案切换运行时行为与回滚
docker	27	引擎不可用降级与容器操作
apihub-transforms	27	配置文本转换的边界与损坏场景
frontmatter	26	SKILL.md 元信息解析
wslmon	25	发行版状态解析与过渡态
registry-v3	17	注册表版本演进与幂等合并

表 5-3 测试密度最高的模块

5.3 缺陷修复记录

开发周期内累计确认并修复缺陷约 40 个（该数字为去重后的确认缺陷数，不含审查 Agent 提出的未证实疑点），分布四类。下表选取其中影响面最大的若干条。

类别	缺陷现象	根因	修复方式
数据损坏	多行 TOML 被写成 CR CR LF	split 与 join 的换行符不匹配	统一换行处理并补回归用例
数据损坏	文本文件被永久写坏	字节级转换前先做	改为纯字节级转换并跳过

		UTF-8 解码	NUL 文件
并发状态	同步操作可重入导致锁失效	锁实现为 Promise 链但未排队	改为共享互斥链并统一入口
边界判定	队列满判断错误导致索引越界	队空与队满共用同一判定	拆分判定逻辑并补边界用例
安全	修复载荷可触发路径穿越写入	载荷由路径名派生	仅传名称，服务端重新推导路径
安全	损坏的注册表被静默清空	解析失败被当作空结果	区分损坏态并拒绝写入

表 5-4 主要缺陷分类与修复方式

表中第一类与第四类的修复存在共同的模式转变：修复动作本身曾被验证为错误。第二行描述的字节级转换改造，是在一轮全量重审中发现原修复会破坏非 UTF-8 文本文件后再次修正的结果——这次二次修复没有新增测试数量，但新增了一条针对 GBK 与 UTF-16 样本的断言。

5.4 交付形态

项目提供两种 Windows 分发形态，均为 x64 架构、约 75 MB：

- NSIS 安装包：一次性安装，启动快，不重复解压约 250 MB 的运行时内容，适合日常使用。
- portable 便携版：免安装单文件，适合多环境验证与快速分发。

两个产物均未做代码签名。安装包图标由 build/icon.png 经多尺寸转换生成并嵌入可执行文件版本信息。

WSL 伴生 CLI 通过 esbuild 打包为零依赖单文件 out/skm.mjs，包含 scan、link、unlink、agents-link、agents-unlink、sync、fix-permissions、selfcheck 八个子命令，始终输出单个 JSON 字符串。它本身不含任何网络代码，但 sync 子命令会调用 git 执行拉取与推送——在默认配置下 origin 指向本地裸仓，因此不需要网络；若把 origin 改为远程地址，它会照常发起网络请求。零依赖单文件这一约束的直接原因是 WSL 侧的 Node 环境需由用户或部署流程预先准备好，任何额外依赖都会引入版本冲突风险。

第六章　结论

SkillVault 的核心命题是：多 Agent 环境下的技能管理必须有一个唯一真相源，且这个真相源应当是本地可版本控制的存储而非云端服务。项目在 13,673 行生产代码与 567 个测试用例的规模上实现了这一命题，并通过双端各自扫描的部署形态解决了 Windows 与 WSL 之间的文件系统边界问题。

从工程角度看，本项目更值得记录的是方法而非成品。在代码由 AI 生成的前提下，项目通过依赖注入、区域所有权标记、可验证的验收流程与人类保留决策权这四道机制，把「AI 生成的代码是否可信」这个问题转化成了可客观检验的问题——每一个交付都可以由独立于交付者的复核者用客观命令重新验证，而不是依赖生成者的自述。这套机制在 12 轮功能增量与多轮全量重审中经受住了检验，累计拦截的缺陷中相当一部分正是自动化测试无法覆盖、而依赖独立复核才能发现的那一类。

项目当前存在明确的未完成边界，已在第 4.4 节列出。这些边界不影响核心命题的成立，但会影响远期可维护性与跨机迁移体验，是后续版本的主要工作方向。